02 · Context: Embeddings, or "Meaning as Location"

The short version

An **embedding** is a list of numbers that stands for a piece of text. It is built so that texts with similar meanings get similar lists. Treat each list as a point (or an arrow) in space, and "find related text" becomes "find nearby points".

Step 1: Why keyword search isn't enough

Classic search matches words. A query for "cheap flights" won't match a document that only says "low-cost airfare", even though they mean the same thing. We want search by **meaning**, not by spelling.

Step 2: Turning text into a point

An **embedding model** (the paper calls it E) reads a text and outputs a fixed-length vector, for example 1,024 numbers. In the toy example we use only 2:

Text	Toy embedding	Reading
"Cat behavior guide"	$[1, 0]$	all cats, no food
"Pizza recipes"	$[0, 1]$	all food, no cats
"Healthy dry food for cats"	$[0.6, 0.8]$	both, leaning food
"cat food" (a query)	$[0.8, 0.6]$	both, leaning cats

Real dimensions don't have clean meanings like "cat-ness", but

the geometry works the same way.

Step 3: "Nearby" means "pointing the same way"

Embeddings are compared by **direction**, using **cosine similarity**: 1 means same direction, 0 means unrelated, −1 means opposite. If every vector is scaled to length 1 first (*normalized*), cosine similarity is just the **dot product**, which computers calculate extremely fast. (The math is in `math/01_math_toolkit.md`.)

Step 4: The index

Before any user asks anything, every document is embedded once and stored. This collection of stored vectors is the **index**. At search time only the *query* needs embedding. The system then compares the query vector with the stored vectors and returns the top few.

A crucial fact for this paper:

> ***The index belongs to the embedding model that built it.*** *Vectors from a different model live in a different space and can't be compared with it. Swap the embedder and you must rebuild the whole index.*

This is why the paper's solution works hard to produce vectors in the **same space** as the existing embedder, so the index can stay as it is.

Step 5: Where embedding models come from

Embedding models are neural networks trained so that matching pairs (a question and its answer, say) land close together and non-matching pairs land far apart. Well-known designs include Sentence-BERT and Dense Passage Retrieval. Many modern ones are built from LLMs (LLM2Vec, NV-Embed, Qwen3-Embedding). An important detail: most embedding models read text **bidirectionally**, meaning every word can see every other word, before and after it. Keep that in mind for file `04`.

Key terms: embedding, embedding model E, vector, cosine similarity, normalization, dot product, index, embedding space.